
Software Requirements Specification

GuessTheWord

Gioco Client-Server a Sfide in Tempo Reale

Corso: Java Programmazione Avanzata

Codice: JA26 **Anno Accademico:** 2025/2026

Dipartimento: DIEM

Università degli Studi di Salerno

Documento	Software Requirements Specification (SRS)
Versione	1.0
Data	Maggio 2025
Standard	IEEE 830-1998 (adattato)
Stato	Definitivo

Questo documento costituisce la specifica formale dei requisiti
per il progetto finale del corso JA26.

Registro delle Revisioni

Versione	Data	Descrizione	Autore
1.0	Maggio 2025	Prima emissione. Documento SRS completo derivato dalle specifiche del progetto JA26.	Gruppo

Contents

Registro delle Revisioni	1
Contents	2
1 Introduzione	4
1.1 Scopo del Documento	4
1.2 Scopo del Sistema	4
1.3 Definizioni, Acronimi e Abbreviazioni	4
1.4 Riferimenti	5
1.5 Panoramica del Documento	5
2 Descrizione Generale	6
2.1 Prospettiva del Sistema	6
2.2 Funzioni Principali del Sistema	6
2.3 Caratteristiche degli Utenti	6
2.4 Vincoli Generali	7
2.5 Assunzioni e Dipendenze	7
3 Requisiti Funzionali	8
3.1 Requisiti del Server	8
3.1.1 Configurazione e Avvio	8
3.1.2 Autenticazione e Gestione Utenti	8
3.1.3 Analisi Documenti	9
3.1.4 Preparazione e Gestione della Sfida	9
3.1.5 Classifiche e Statistiche (Lato Server)	10
3.2 Requisiti del Client	10
3.2.1 Configurazione	11
3.2.2 Autenticazione	11
3.2.3 Finestra di Gioco	11
3.2.4 Storico e Classifiche (Lato Client)	12
3.3 Requisiti di Persistenza dei Dati	12
4 Requisiti Non Funzionali	14
4.1 Usabilità	14
4.2 Affidabilità	14
4.3 Performance	14
4.4 Manutenibilità	15
4.5 Portabilità	15
5 Requisiti di Sistema e Vincoli Tecnologici	16
5.1 Tecnologie Obbligatorie	16
5.2 Tecnologie Facoltative	16
5.3 Ambiente di Esecuzione	16
5.4 Struttura dell'Archivio di Esecuzione	16
6 Modello dei Casi d'Uso	17
6.1 Attori del Sistema	17

6.2	Casi d’Uso — Amministratore	17
6.3	Casi d’Uso — Giocatore	17
6.4	Caso d’Uso Dettagliato — UC-G04: Partecipazione alla Sfida	18
7	Schema del Database	19
7.1	Tabella: users	19
7.2	Tabella: challenges	19
7.3	Tabella: results	19
7.4	Query di Esempio per le Statistiche	19
8	Requisiti di Consegna e Valutazione	21
8.1	Materiale Obbligatorio	21
8.2	Scadenza	21
8.3	Requisiti per l’Archivio Eseguitibile	22
8.4	Modalità di Valutazione — Esame Finale	22
9	Appendici	23
9.1	Appendice A — Riepilogo dei Requisiti	23
9.1.1	Requisiti Funzionali	23
9.1.2	Requisiti Non Funzionali	23
9.2	Appendice B — Protocollo di Comunicazione (Schema Concettuale)	24
9.3	Appendice C — Algoritmo del Cifrario di Cesare	24
9.4	Appendice D — Checklist di Verifica Pre-Consegna	25

1 Introduzione

1.1 Scopo del Documento

Il presente documento costituisce la **Software Requirements Specification (SRS)** del sistema *GuessTheWord*, sviluppato come progetto finale per il Corso di Java Programmazione Avanzata (JA26), A.A. 2025/2026, presso il Dipartimento DIEM dell'Università degli Studi di Salerno.

L'obiettivo di questo documento è fornire una descrizione completa, precisa e non ambigua dei requisiti funzionali, non funzionali e di sistema del software, seguendo le linee guida dello standard IEEE 830-1998. Il documento è destinato al team di sviluppo, al docente valutatore e a chiunque partecipi al processo di revisione del progetto.

1.2 Scopo del Sistema

GuessTheWord è un **gioco multiplayer client-server** sviluppato interamente in Java, con interfaccia grafica realizzata in JavaFX. Due utenti si sfidano in tempo reale nel decifrare una o più parole nascoste all'interno di un estratto testuale. Le parole da indovinare vengono cifrate tramite il **cifrario di Cesare** applicato con uno spostamento scelto casualmente dal server.

Il sistema è composto da tre componenti principali:

- **Server JavaFX**: coordina l'intero sistema, gestisce le connessioni, l'autenticazione, le sfide e la persistenza dei dati.
- **Client JavaFX** (due istanze): forniscono l'interfaccia utente per l'autenticazione, la partecipazione alle sfide e la visualizzazione dello storico.
- **Database SQLite**: garantisce la persistenza di credenziali, sfide e risultati.

1.3 Definizioni, Acronimi e Abbreviazioni

Termine Acronimo	Definizione
SRS	Software Requirements Specification
JA26	Codice identificativo del corso
JavaFX	Framework Java per la creazione di interfacce grafiche desktop
FXML	File XML usato per descrivere layout JavaFX
Socket	Meccanismo di comunicazione di rete TCP/IP
SQLite	Database relazionale leggero, file-based
JDBC	Java Database Connectivity, API standard per accesso a DB in Java
Stream API	API Java 8+ per l'elaborazione funzionale di sequenze di dati
Service/Task	Classi JavaFX per esecuzione asincrona di operazioni in background

Termine / Acronimo	Definizione
Cifrario di Cesare	Tecnica di cifratura per sostituzione di caratteri con uno spostamento fisso
TF	Term Frequency: frequenza relativa di una parola in un documento
Shift	Valore numerico dello spostamento nel cifrario di Cesare
Admin	Utente amministratore con privilegi speciali di gestione del server
Sfida	Una partita tra due utenti registrati, coordinata dal server
Estratto	Porzione di testo selezionata dal server per una sfida
properties	File di configurazione in formato chiave=valore
JAR	Java ARchive, formato eseguibile Java
JDK	Java Development Kit
RF	Requisito Funzionale
RNF	Requisito Non Funzionale
RC	Requisito di Consegna

1.4 Riferimenti

- Specifiche del Progetto Finale — Corso JA26, A.A. 2025/2026, Dipartimento DIEM, UNISA.
- IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications*.
- Documentazione ufficiale Java 8 — Oracle Corporation.
- Documentazione ufficiale JavaFX — OpenJFX Project.
- SQLite Documentation — <https://www.sqlite.org/docs.html>.

1.5 Panoramica del Documento

Il documento è strutturato come segue:

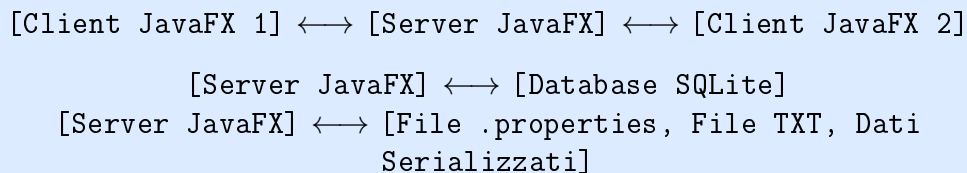
- **Sezione 1 – Introduzione:** scopo, definizioni e panoramica.
- **Sezione 2 – Descrizione Generale:** contesto, assunzioni e dipendenze.
- **Sezione 3 – Requisiti Funzionali:** funzionalità dettagliate del sistema.
- **Sezione 4 – Requisiti Non Funzionali:** qualità, performance, sicurezza.
- **Sezione 5 – Requisiti di Sistema e Vincoli Tecnologici.**
- **Sezione 6 – Modello dei Casi d’Uso:** attori e use case principali.
- **Sezione 7 – Schema del Database.**
- **Sezione 8 – Requisiti di Consegna e Valutazione.**
- **Sezione 9 – Appendici.**

2 Descrizione Generale

2.1 Prospettiva del Sistema

GuessTheWord è un sistema software **autonomo e auto-contenuto**, progettato per funzionare interamente in locale (localhost) o su rete LAN. Non dipende da servizi cloud, API esterne o infrastrutture di terze parti.

Il sistema segue un'architettura **client-server** classica:



La comunicazione tra client e server avviene tramite **socket TCP**. Il server è l'unico componente che accede al database e che conosce le risposte corrette alle sfide.

2.2 Funzioni Principali del Sistema

Il sistema svolge le seguenti macro-funzioni:

1. **Gestione utenti:** registrazione, autenticazione, distinzione di ruolo (admin/giocatore).
2. **Analisi documenti:** caricamento di file TXT, calcolo della Term Frequency con Stream API, salvataggio serializzato dei risultati.
3. **Preparazione sfide:** selezione di estratti testuali, cifratura con Cesare, invio ai client.
4. **Gestione partite:** coordinamento in tempo reale, validazione risposte, dichiarazione vincitore.
5. **Persistenza e classifiche:** storico sfide, statistiche per utente, consultazione classifiche.
6. **Configurazione:** lettura da file `.properties` per IP, porta e altri parametri.

2.3 Caratteristiche degli Utenti

Ruolo	Descrizione	Competenze Richieste
Amministratore	Utente speciale con accesso all'interfaccia di gestione del server. Gestisce i documenti e monitora il sistema.	Conoscenza di base del sistema; le sue credenziali sono pre-caricate nel DB.
Giocatore	Utente che si registra o accede tramite il client e partecipa alle sfide.	Nessuna competenza tecnica richiesta; interazione esclusivamente tramite GUI.

2.4 Vincoli Generali

Vincolo

- Il sistema **DEVE** essere compatibile con **JDK 8**.
- Il sistema **DEVE** utilizzare esclusivamente le tecnologie obbligatorie elencate nella Sezione 5.
- Tutti i percorsi di file e database **DEVE** essere **relativi**, mai assoluti.
- Nessuna dipendenza esterna **DEVE** essere richiesta al di fuori di quelle incluse nell'archivio di consegna.
- Il sistema **DEVE** essere avviabile tramite semplice esecuzione di file JAR.

2.5 Assunzioni e Dipendenze

- Si assume che il sistema operativo host supporti Java 8 e JavaFX 8.
- Si assume che il server e i client vengano avviati sulla stessa macchina (localhost) o in rete LAN durante i test.
- Si assume che il database SQLite venga inizializzato automaticamente all'avvio con lo schema e gli account predefiniti.
- Si assume che i file TXT da analizzare siano già disponibili nel filesystem prima dell'avvio del server.
- La disponibilità della rete è necessaria per la comunicazione via socket tra client e server.

3 Requisiti Funzionali

I requisiti sono classificati con il seguente schema di priorità:

- **[MUST]**: requisito obbligatorio, indispensabile per il funzionamento del sistema.
- **[SHOULD]**: requisito fortemente consigliato.
- **[MAY]**: requisito opzionale o facoltativo.

3.1 Requisiti del Server

3.1.1 Configurazione e Avvio

RF-SRV-01 Lettura configurazione da file [MUST]

Il server **DEVE** leggere la porta di ascolto da un file di configurazione denominato `server.properties`, nella forma:

```
server.port=5000
```

Il file **DEVE** essere localizzabile tramite percorso relativo alla directory di esecuzione.

RF-SRV-02 Apertura ServerSocket [MUST]

Il server **DEVE** aprire un `ServerSocket` sulla porta specificata nel file di configurazione e rimanere in ascolto di connessioni client in modo continuo.

RF-SRV-03 Interfaccia grafica amministratore [MUST]

Il server **DEVE** fornire un'interfaccia grafica JavaFX (con file FXML) accessibile all'amministratore, che consenta di:

- selezionare uno o più file di testo (`.txt`) da analizzare;
- avviare l'analisi del contenuto;
- salvare i risultati dell'analisi in formato serializzato;
- caricare risultati di analisi precedentemente salvati.

3.1.2 Autenticazione e Gestione Utenti

RF-SRV-04 Autenticazione client [MUST]

Il server **DEVE** ricevere le credenziali (username e password) inviate dai client, verificarle nel database SQLite e rispondere con l'esito dell'autenticazione.

RF-SRV-05 Registrazione nuovi utenti [MUST]

Il server **DEVE** ricevere richieste di registrazione dai client, verificare l'unicità dello username e, in caso di successo, inserire il nuovo utente nel database con il ruolo di *giocatore*.

RF-SRV-06 Gestione account amministratore [MUST]

Il server **DEVE** riconoscere l'account amministratore e consentirne l'accesso all'interfaccia di gestione. Le credenziali dell'amministratore **DEVE** essere pre-caricate nel database.

3.1.3 Analisi Documenti**RF-SRV-07 Analisi Term Frequency [MUST]**

Il server **DEVE** elaborare i documenti TXT selezionati calcolando la **Term Frequency** (frequenza relativa) di ciascuna parola. L'elaborazione **DEVE** utilizzare la **Stream API di Java**.

RF-SRV-08 Esecuzione asincrona dell'analisi [MUST]

L'analisi dei documenti **DEVE** essere eseguita in modo asincrono tramite **JavaFX Service / Task**, in modo che l'interfaccia grafica del server rimanga reattiva durante l'elaborazione.

RF-SRV-09 Salvataggio serializzato dei risultati [MUST]

Il server **DEVE** consentire il salvataggio dei risultati dell'analisi (mappa parola → frequenza) in formato **serializzato** su file, e il successivo ricaricamento all'avvio, evitando di rieseguire l'analisi.

3.1.4 Preparazione e Gestione della Sfida**RF-SRV-10 Attesa di due client connessi e autenticati [MUST]**

Il server **DEVE** avviare una sfida solamente quando **entrambi i client** si sono connessi e hanno completato con successo l'autenticazione.

RF-SRV-11 Selezione dell'estratto testuale [MUST]

Il server **DEVE** selezionare un estratto di testo da uno dei documenti analizzati. La selezione dell'estratto **DOVREBBE** avvenire in modo da garantire varietà tra le sfide.

RF-SRV-12 Applicazione del cifrario di Cesare [MUST]

Il server **DEVE** selezionare una o più parole dall'estratto e sostituirle con la rispettiva versione cifrata tramite il **cifrario di Cesare**, applicando uno spostamento scelto **casualmente**. Le versioni cifrate **DEVE** essere chiaramente distinguibili nel testo inviato ai client.

RF-SRV-13 Invio del testo cifrato ai client [MUST]

Il server **DEVE** inviare a entrambi i client:

- il testo dell'estratto con le parole cifrate;
- la durata del conto alla rovescia (timeout della sfida).

RF-SRV-14 Ricezione e validazione delle risposte [MUST]

Il server **DEVE** ricevere le risposte inviate dai client, validarle confrontandole con la parola originale (in modo case-insensitive o secondo le regole stabilite) e determinare il vincitore come il **primo giocatore che risponde correttamente**.

RF-SRV-15 Notifica dell'esito [MUST]

Il server **DEVE** notificare **entrambi** i client dell'esito della sfida, indicando chi ha vinto e, se applicabile, la risposta corretta.

RF-SRV-16 Registrazione dell'esito nel database [MUST]

Al termine di ogni sfida, il server **DEVE** registrare nel database:

- gli utenti partecipanti;
- l'esito (vincitore e perdente);
- il riferimento temporale (data e ora della sfida).

RF-SRV-17 Livelli di difficoltà (opzionale) [MAY]

Il server **PUÒ** implementare livelli di difficoltà variabili per le sfide, basati su:

- frequenza della parola selezionata (parola più rara = difficoltà maggiore);
- lunghezza della parola;
- numero di parole cifrate simultaneamente;
- valore dello shift del cifrario.

3.1.5 Classifiche e Statistiche (Lato Server)

RF-SRV-18 Visualizzazione classifiche dall'interfaccia admin [SHOULD]

Il server **DOVREBBE** consentire all'amministratore di visualizzare classifiche e statistiche sulle partite giocate tramite l'interfaccia grafica, inclusi:

- numero di vittorie per utente;
- numero totale di partite disputate per utente;
- tempo medio di risposta.

3.2 Requisiti del Client

3.2.1 Configurazione

RF-CLT-01 Lettura configurazione da file [MUST]

Ciascun client **DEVE** leggere l'indirizzo IP e la porta del server da un file di configurazione denominato `client.properties`, nella forma:

```
server.ip=127.0.0.1
server.port=5000
```

3.2.2 Autenticazione

RF-CLT-02 Interfaccia di login e registrazione [MUST]

Il client **DEVE** presentare una finestra iniziale (JavaFX/FXML) con:

- campo di inserimento per lo username;
- campo di inserimento per la password;
- pulsante *Login*;
- pulsante *Registrazione*.

RF-CLT-03 Comunicazione credenziali al server [MUST]

Il client **DEVE** trasmettere al server le credenziali inserite dall'utente e riceverne l'esito, informando l'utente in caso di errore (credenziali errate, username già esistente, ecc.).

RF-CLT-04 Attesa dell'avversario [MUST]

Dopo l'autenticazione, il client **DEVE** mostrare uno stato di **attesa** finché l'altro sfidante non si connette e autentica.

3.2.3 Finestra di Gioco

RF-CLT-05 Visualizzazione del testo cifrato [MUST]

Durante la sfida, il client **DEVE** visualizzare il testo ricevuto dal server con le parole cifrate **chiaramente evidenziate** o distinguibili dal resto del testo.

RF-CLT-06 Inserimento e invio della risposta [MUST]

Il client **DEVE** fornire:

- un campo di testo per l'inserimento della risposta;
- un pulsante per l'invio della risposta al server.

RF-CLT-07 Timer visibile [MUST]

Il client **DEVE** mostrare un **conto alla rovescia** visibile, sincronizzato con la durata della sfida comunicata dal server.

RF-CLT-08 Indicatore di stato della partita [MUST]

Il client **DEVE** mostrare lo stato attuale della partita, che include almeno i seguenti stati:

- *In attesa* — connesso, in attesa dell'avversario;
- *In gioco* — sfida in corso;
- *Terminata* — sfida conclusa, esito ricevuto.

RF-CLT-09 Visualizzazione esito [MUST]

Il client **DEVE** visualizzare l'esito della sfida ricevuto dal server, indicando se l'utente ha vinto o perso e la risposta corretta.

3.2.4 Storico e Classifiche (Lato Client)**RF-CLT-10 Visualizzazione storico personale [SHOULD]**

Il client **DOVREBBE** consentire a ciascun utente di visualizzare lo storico delle proprie sfide precedenti, con esiti e date.

3.3 Requisiti di Persistenza dei Dati**RF-DB-01 Tabella Utenti [MUST]**

Il database **DEVE** contenere una struttura dati per gli utenti, che includa almeno:

- identificativo univoco dell'utente;
- username (univoco);
- password (si raccomanda hashing);
- ruolo (amministratore / giocatore).

RF-DB-02 Tabella Sfide [MUST]

Il database **DEVE** contenere una struttura dati per le sfide disputate, che includa almeno:

- identificativo univoco della sfida;
- riferimento temporale (data e ora);
- testo dell'estratto e parola/e cifrata/e (opzionale).

RF-DB-03 Tabella Risultati [MUST]

Il database **DEVE** contenere una struttura dati per i risultati, che includa almeno:

- riferimento alla sfida;
- riferimento agli utenti partecipanti;
- esito per ciascun utente (vittoria / sconfitta);
- tempo di risposta (opzionale).

RF-DB-04 Statistiche estraibili [MUST]

Il sistema **DEVE** supportare l'estrazione delle seguenti statistiche tramite query sul database:

- numero di vittorie per utente;
- numero di partite disputate per utente;
- tempo medio di risposta per utente.

RF-DB-05 Account predefiniti [MUST]

Il database **DEVE** contenere, all'avvio del sistema, almeno i seguenti account pre-caricati:

- **1 account amministratore;**
- **2 account giocatore.**

Le credenziali di tali account **DEVE** essere documentate nella relazione e nel file `readme.txt`.

4 Requisiti Non Funzionali

4.1 Usabilità

RNF-USA-01 Interfaccia intuitiva [MUST]

Le interfacce grafiche (server e client) **DEVE** essere intuitive e navigabili senza necessità di documentazione aggiuntiva per un utente con minime competenze informatiche.

RNF-USA-02 Feedback visivo [MUST]

Il sistema **DEVE** fornire feedback visivo per ogni azione significativa dell'utente (login, invio risposta, fine partita, ecc.).

RNF-USA-03 Styling CSS (opzionale) [MAY]

Le interfacce JavaFX **PUÒ** essere stilizzate tramite fogli di stile CSS per migliorare l'aspetto grafico.

4.2 Affidabilità

RNF-AFF-01 Gestione disconnessioni [SHOULD]

Il server **DOVREBBE** gestire in modo robusto la disconnessione inattesa di un client durante una sfida, notificando l'altro client e aggiornando lo stato della partita nel database.

RNF-AFF-02 Validazione input [MUST]

Il sistema **DEVE** validare tutti gli input utente (campi vuoti, caratteri non validi) sia lato client (prima dell'invio) sia lato server (prima dell'elaborazione).

4.3 Performance

RNF-PRF-01 Reattività dell'interfaccia [MUST]

L'interfaccia grafica del server **DEVE** rimanere reattiva durante l'analisi dei documenti, grazie all'uso di **JavaFX Service / Task**.

RNF-PRF-02 Latenza di comunicazione [SHOULD]

La latenza di comunicazione tra client e server, in condizioni di rete locale (local-host / LAN), **DOVREBBE** essere sufficientemente bassa da non compromettere l'esperienza di gioco.

4.4 Manutenibilità

RNF-MNT-01 Documentazione Javadoc [MUST]

Il codice sorgente **DEVE** essere documentato con **Javadoc** per tutte le classi e i metodi pubblici.

RNF-MNT-02 Configurabilità [MUST]

Tutti i parametri configurabili (porta, IP, ecc.) **DEVE** essere gestiti tramite file `.properties` e non codificati direttamente nel sorgente (*no hard-coding*).

4.5 Portabilità

RNF-PRT-01 Compatibilità JDK 8 [MUST]

Tutti i file JAR prodotti **DEVE** essere compatibili con **JDK 8** e non richiedere versioni successive.

RNF-PRT-02 Dipendenze auto-contenute [MUST]

L'archivio di consegna **DEVE** includere tutte le dipendenze necessarie. Non **DEVE** essere richiesta nessuna installazione aggiuntiva da parte del revisore.

RNF-PRT-03 Percorsi relativi [MUST]

Tutti i percorsi a file, database e risorse **DEVE** essere **relativi** alla directory di esecuzione, mai assoluti.

5 Requisiti di Sistema e Vincoli Tecnologici

5.1 Tecnologie Obbligatorie

Le seguenti tecnologie **DEVE** essere utilizzate nel progetto:

Tecnologia	Utilizzo nel Progetto
Java (JDK 8)	Linguaggio principale per server e client
JavaFX + FXML	Interfaccia grafica per server (admin) e client (giocatori)
Socket TCP	Comunicazione client-server in tempo reale
Stream API	Elaborazione del testo per l'analisi della Term Frequency
JavaFX Service / Task	Esecuzione asincrona dell'analisi documenti
SQLite (via JDBC)	Persistenza di utenti, sfide e risultati
File .properties	Configurazione di server e client
Serializzazione Java	Salvataggio e ricaricamento dei risultati dell'analisi

5.2 Tecnologie Facoltative

Tecnologia	Utilizzo nel Progetto
CSS (JavaFX)	Styling delle interfacce grafiche

5.3 Ambiente di Esecuzione

- Sistema Operativo: qualsiasi sistema supportato da JDK 8 (Windows, macOS, Linux).
- JDK 8 o superiore installato sulla macchina del revisore.
- Accesso alla rete locale (socket su localhost o LAN).

5.4 Struttura dell'Archivio di Esecuzione

L'archivio eseguibili.zip **DEVE** avere la seguente struttura minima:

eseguibili/
server.jar
client.jar
readme.txt
properties/
server.properties
client.properties
db/
database.db
documents/
data/

I file possono essere organizzati in modo diverso purché tutti i componenti necessari siano presenti e il sistema sia immediatamente eseguibile.

6 Modello dei Casi d'Uso

6.1 Attori del Sistema

Attore	Descrizione
Amministratore	Utente speciale che gestisce il server tramite l'interfaccia admin. Carica i documenti, avvia le analisi e monitora le classifiche.
Giocatore	Utente registrato che si connette tramite il client per partecipare alle sfide.
Server	Componente software attivo che coordina le connessioni, le sfide e la persistenza. Attore secondario nei casi d'uso del client.

6.2 Casi d'Uso — Amministratore

ID	Nome	Descrizione
UC-A01	Login amministratore	L'amministratore accede al server con le proprie credenziali tramite l'interfaccia admin.
UC-A02	Caricamento documenti	L'amministratore seleziona uno o più file TXT da filesystem per l'analisi.
UC-A03	Avvio analisi	L'amministratore avvia l'analisi TF dei documenti caricati; la UI rimane reattiva durante l'elaborazione.
UC-A04	Salvataggio analisi	L'amministratore salva i risultati dell'analisi in formato serializzato.
UC-A05	Caricamento analisi	L'amministratore carica i risultati di un'analisi precedentemente salvata.
UC-A06	Visualizzazione classifiche	L'amministratore consulta le statistiche e le classifiche degli utenti.

6.3 Casi d'Uso — Giocatore

ID	Nome	Descrizione
UC-G01	Registrazione	Il giocatore crea un nuovo account inserendo username e password. Il server verifica l'unicità e registra l'utente.
UC-G02	Login	Il giocatore accede con le proprie credenziali. Il server autentica e conferma l'accesso.
UC-G03	Attesa partita	Dopo il login, il giocatore attende che il secondo client si connetta e autentichi.
UC-G04	Partecipazione sfida	Il giocatore riceve il testo cifrato, lo analizza e invia la propria risposta entro il tempo limite.

ID	Nome	Descrizione
UC-G05	Visualizzazione esito	Al termine della sfida, il giocatore visualizza se ha vinto o perso e la parola corretta.
UC-G06	Visualizzazione storico	Il giocatore consulta lo storico delle proprie sfide precedenti con relativi esiti.

6.4 Caso d'Uso Dettagliato — UC-G04: Partecipazione alla Sfida

UC-G04 — Partecipazione alla Sfida

Attore Primario: Giocatore

Attori Secondari: Server, secondo Giocatore

Pre-condizioni:

- Entrambi i giocatori sono autenticati e connessi al server.
- Il server ha almeno un documento analizzato disponibile.

Flusso Principale:

1. Il server seleziona un estratto da un documento analizzato.
2. Il server sceglie una o più parole e le cifra con Cesare (shift casuale).
3. Il server invia il testo cifrato e il timeout a entrambi i client.
4. I client visualizzano il testo con il timer attivo.
5. I giocatori inseriscono e inviano la loro risposta.
6. Il server riceve le risposte e valida.
7. Il primo giocatore con risposta corretta viene dichiarato vincitore.
8. Il server notifica entrambi i client e registra l'esito nel database.

Flusso Alternativo A — Timeout:

- Se nessun giocatore risponde correttamente entro il timeout, la sfida termina in pareggio o senza vincitore.
- Il server notifica entrambi i client e registra l'esito.

Post-condizioni:

- L'esito della sfida è registrato nel database.
- Entrambi i client mostrano il risultato finale.

7 Schema del Database

Il design del database è lasciato alla libera progettazione del gruppo. La struttura seguente rappresenta un modello di riferimento che soddisfa i requisiti descritti nel documento.

7.1 Tabella: users

Colonna	Tipo	Descrizione
user_id	INTEGER PK	Chiave primaria auto-incrementale
username	TEXT UNIQUE	Nome utente univoco
password_hash	TEXT	Hash della password
role	TEXT	Ruolo: <code>admin</code> o <code>player</code>
created_at	DATETIME	Data e ora di registrazione

7.2 Tabella: challenges

Colonna	Tipo	Descrizione
challenge_id	INTEGER PK	Chiave primaria auto-incrementale
played_at	DATETIME	Data e ora della sfida
text_excerpt	TEXT	Estratto di testo usato
encrypted_word	TEXT	Parola/e cifrata/e
original_word	TEXT	Parola/e originale/i
caesar_shift	INTEGER	Spostamento del cifrario applicato

7.3 Tabella: results

Colonna	Tipo	Descrizione
result_id	INTEGER PK	Chiave primaria auto-incrementale
challenge_id	INTEGER FK	Riferimento alla sfida
user_id	INTEGER FK	Riferimento all'utente
outcome	TEXT	Esito: <code>win</code> / <code>loss</code> / <code>draw</code>
response_time_ms	INTEGER	Tempo di risposta in millisecondi

7.4 Query di Esempio per le Statistiche

```
SELECT u.username, COUNT(*) AS wins
FROM results r
JOIN users u ON r.user_id = u.user_id
WHERE r.outcome = 'win'
GROUP BY r.user_id
```

```
ORDER BY wins DESC;
```

Listing 1: Numero di vittorie per utente

```
SELECT u.username ,  
       COUNT(*) AS games_played ,  
       AVG(r.response_time_ms) AS avg_response_ms  
FROM results r  
JOIN users u ON r.user_id = u.user_id  
GROUP BY r.user_id  
ORDER BY avg_response_ms ASC;
```

Listing 2: Tempo medio di risposta per utente

8 Requisiti di Consegna e Valutazione

8.1 Materiale Obbligatorio

RC-01 Codice sorgente [MUST]

DEVE essere consegnato il codice sorgente completo del progetto (moduli server e client), corredato di **Javadoc** per tutte le classi e i metodi pubblici.

RC-02 Relazione tecnica [MUST]

DEVE essere prodotta una relazione sintetica che documenti:

- Architettura del sistema;
- Principali scelte progettuali;
- Specifica dei requisiti (Use Cases, Mockup interfacce);
- Progettazione (Class Diagram, Sequence Diagram, Modello E-R);
- Tecnologie utilizzate.

RC-03 Presentazione [MUST]

DEVE essere prodotta una **presentazione PowerPoint** sintetica del progetto.

RC-04 Archivio eseguibili [MUST]

DEVE essere consegnato un archivio ZIP denominato `eseguibili.zip` contenente:

- `server.jar`
- `client.jar`
- `readme.txt` (con credenziali degli account predefiniti)
- File di configurazione (`server.properties`, `client.properties`)
- Database SQLite inizializzato (`database.db`)
- Tutte le dipendenze necessarie

8.2 Scadenza

Vincolo

La consegna del progetto **DEVE** avvenire **7 giorni prima dell'appello d'esame**.

8.3 Requisiti per l'Archivio Eseguibile

RC-05 Avvio immediato [MUST]

Il revisore **DEVE** poter avviare il sistema con i seguenti comandi, senza ulteriori configurazioni:

```
java -jar server.jar
java -jar client.jar
java -jar client.jar
```

RC-06 Account predefiniti per il testing [MUST]

Il database consegnato **DEVE** contenere:

- **1 account amministratore** (per accesso all'interfaccia admin del server);
- **2 account giocatore** (per testare immediatamente una sfida completa).

Le credenziali **DEVE** essere riportate sia nella relazione sia nel file `readme.txt`.

8.4 Modalità di Valutazione — Esame Finale

- L'esame consiste in una **presentazione di gruppo** (circa 10 minuti), comprensiva di **demo live** del progetto.
- Segue un **colloquio individuale** sulle scelte progettuali e sugli argomenti del corso.
- Tutti i componenti del gruppo **DEVE** dimostrare padronanza dell'intero progetto.
- Il colloquio individuale non è limitato al progetto: può riguardare qualsiasi argomento trattato durante il corso.
- La consegna di archivi incompleti o non eseguibili influisce negativamente sulla valutazione.
- È preferibile che tutti i membri del gruppo sostengano l'esame nello stesso appello.
- Gli studenti che rimandano l'esame **DEVE** ripresentare il progetto nell'appello successivo.

9 Appendici

9.1 Appendice A — Riepilogo dei Requisiti

9.1.1 Requisiti Funzionali

ID	Descrizione sintetica	Componente	Priorità
RF-SRV-01	Lettura porta da server.properties	Server	MUST
RF-SRV-02	Apertura ServerSocket	Server	MUST
RF-SRV-03	Interfaccia admin JavaFX	Server	MUST
RF-SRV-04	Autenticazione client	Server	MUST
RF-SRV-05	Registrazione nuovi utenti	Server	MUST
RF-SRV-06	Gestione account admin	Server	MUST
RF-SRV-07	Analisi TF con Stream API	Server	MUST
RF-SRV-08	Analisi asincrona con Service/Task	Server	MUST
RF-SRV-09	Serializzazione risultati analisi	Server	MUST
RF-SRV-10	Attesa due client autenticati	Server	MUST
RF-SRV-11	Selezione estratto testuale	Server	MUST
RF-SRV-12	Cifrario di Cesare con shift casuale	Server	MUST
RF-SRV-13	Invio testo cifrato ai client	Server	MUST
RF-SRV-14	Validazione risposte	Server	MUST
RF-SRV-15	Notifica esito a entrambi i client	Server	MUST
RF-SRV-16	Registrazione esito nel DB	Server	MUST
RF-SRV-17	Livelli di difficoltà	Server	MAY
RF-SRV-18	Classifiche nell'interfaccia admin	Server	SHOULD
RF-CLT-01	Lettura IP/porta da client.properties	Client	MUST
RF-CLT-02	Interfaccia login/registrazione	Client	MUST
RF-CLT-03	Comunicazione credenziali al server	Client	MUST
RF-CLT-04	Attesa avversario	Client	MUST
RF-CLT-05	Visualizzazione testo cifrato	Client	MUST
RF-CLT-06	Inserimento e invio risposta	Client	MUST
RF-CLT-07	Timer visibile	Client	MUST
RF-CLT-08	Indicatore stato partita	Client	MUST
RF-CLT-09	Visualizzazione esito	Client	MUST
RF-CLT-10	Storico personale	Client	SHOULD
RF-DB-01	Struttura tabella Utenti	Database	MUST
RF-DB-02	Struttura tabella Sfide	Database	MUST
RF-DB-03	Struttura tabella Risultati	Database	MUST
RF-DB-04	Statistiche estraibili	Database	MUST
RF-DB-05	Account predefiniti nel DB	Database	MUST

9.1.2 Requisiti Non Funzionali

ID	Descrizione sintetica	Categoria	Priorità
RNF-USA-01	Interfaccia intuitiva	Usabilità	MUST
RNF-USA-02	Feedback visivo	Usabilità	MUST

ID	Descrizione sintetica	Categoria	Priorità
RNF-USA-03	Styling CSS	Usabilità	MAY
RNF-AFF-01	Gestione disconnessioni	Affidabilità	SHOULD
RNF-AFF-02	Validazione input	Affidabilità	MUST
RNF-PRF-01	Reattività UI durante analisi	Performance	MUST
RNF-PRF-02	Bassa latenza in LAN	Performance	SHOULD
RNF-MNT-01	Documentazione Javadoc	Manutenibilità	MUST
RNF-MNT-02	Configurabilità tramite .properties	Manutenibilità	MUST
RNF-PRT-01	Compatibilità JDK 8	Portabilità	MUST
RNF-PRT-02	Dipendenze auto-contenute	Portabilità	MUST
RNF-PRT-03	Percorsi relativi	Portabilità	MUST

9.2 Appendice B — Protocollo di Comunicazione (Schema Concettuale)

Di seguito è riportato uno schema concettuale del protocollo di comunicazione tra client e server (la definizione dettagliata dei messaggi è lasciata al gruppo):

```

CLIENT --> SERVER : LOGIN_REQUEST { username, password }
SERVER --> CLIENT : LOGIN_RESPONSE { success | fail, message }

CLIENT --> SERVER : REGISTER_REQUEST { username, password }
SERVER --> CLIENT : REGISTER_RESPONSE { success | fail, message }

-- Quando entrambi i client sono autenticati --

SERVER --> CLIENT : CHALLENGE_START {
    text_excerpt,
    encrypted_words,
    timeout_seconds
}

CLIENT --> SERVER : CHALLENGE_ANSWER { word_guess }

SERVER --> CLIENT : CHALLENGE_RESULT {
    winner_username,
    correct_word,
    outcome (win | loss | draw)
}

```

Listing 3: Flusso messaggi client-server

9.3 Appendice C — Algoritmo del Cifrario di Cesare

Il cifrario di Cesare sostituisce ogni lettera dell'alfabeto con la lettera che si trova a uno spostamento (shift) k posizioni avanti nell'alfabeto. La formula per la cifratura è:

$$E(c) = (c - 'A' + k)26 + 'A'$$

Dove c è il carattere originale e k è lo shift casuale scelto dal server ($1 \leq k \leq 25$). La decifrazione usa lo shift inverso $26 - k$.

```
public static String encrypt(String word, int shift) {
    StringBuilder sb = new StringBuilder();
    for (char c : word.toCharArray()) {
        if (Character.isLetter(c)) {
            char base = Character.isUpperCase(c) ? 'A' : 'a';
            sb.append((char) ((c - base + shift) % 26 + base));
        } else {
            sb.append(c);
        }
    }
    return sb.toString();
}

public static String decrypt(String word, int shift) {
    return encrypt(word, 26 - shift);
}
```

Listing 4: Esempio di implementazione Java del cifrario di Cesare

9.4 Appendice D — Checklist di Verifica Pre-Consegna

N.	Elemento da verificare	OK?
1	Codice sorgente completo per server e client	
2	Javadoc presente per tutte le classi e metodi pubblici	
3	Relazione tecnica con architettura, Use Case, Mockup, Diagrammi	
4	Presentazione PowerPoint preparata	
5	<code>server.jar</code> incluso nell'archivio	
6	<code>client.jar</code> incluso nell'archivio	
7	File <code>server.properties</code> incluso e correttamente configurato	
8	File <code>client.properties</code> incluso e correttamente configurato	
9	Database SQLite pre-inizializzato con schema e account predefiniti	
10	File <code>readme.txt</code> con credenziali degli account predefiniti	
11	Tutte le dipendenze incluse nell'archivio	
12	Percorsi di file e DB sono relativi (nessun percorso assoluto)	
13	JAR compatibile con JDK 8	
14	Avvio con <code>java -jar server.jar</code> funzionante senza configurazioni extra	
15	Avvio di due istanze <code>java -jar client.jar</code> funzionante	
16	Test completo di una sfida tra due client eseguito con successo	
17	Archivio denominato correttamente <code>eseguibili.zip</code>	
18	Consegna effettuata almeno 7 giorni prima dell'appello	